
You:

Wait let me understand rasterio better

ChatGPT:

Absolutely. Let's pause terrain cost modelling and go back to Rasterio.

I agree with that decision. Before we build slope calculations and A*, you should understand how we get information out of the MOLA GeoTIFF. Otherwise, we'll end up writing routing code without understanding where its data comes from.

Let's establish what you need to understand

We'll learn Rasterio in this order:

1. **Opening a raster** — what `rasterio.open()` does and what a dataset object represents.
2. **Raster metadata** — width, height, bands, CRS, bounds, resolution and transform.
3. **Reading raster values** — what `read(1)` returns and how pixels are indexed.
4. **Raster windows** — extracting a small geographical region without loading the global dataset.
5. **Coordinate conversion** — converting Mars latitude/longitude into raster row and column.
6. **NoData and data validity** — understanding missing or invalid elevation values.
7. **Elevation lookup** — building a function that retrieves elevation at a given coordinate.

Where we left off

You already inspected your USGS MOLA dataset and obtained:

Property	Your dataset
Width	46,080 pixels
Height	23,040 pixels
Bands	1
Resolution	~463.09 m
Data type	int16
NoData	-32768
CRS	Mars Equirectangular

These aren't just technical details. Each one tells us something about how Marswalk can use the dataset.

Let's start with one fundamental question

When you write:

```
import rasterio

dataset = rasterio.open("mola-global.tif")
```

What do you think `dataset` represents?

- A. The complete elevation data loaded into memory.
- B. An object that gives us access to the raster file and its information.
- C. A two-dimensional Python list containing elevation values.

I want to check your understanding before moving forward. No rush—we'll take Rasterio one concept at a time.

You:

B

ChatGPT:

(Correct!)

B — `dataset` is an object that gives us access to the raster file and its information.

This is an important distinction.

When you write:

```
dataset = rasterio.open("mola-global.tif")
```

Rasterio does not immediately load all 1.06 billion elevation cells into RAM.

Instead, it opens the file and gives you a **DatasetReader object**.

Think of it like opening a book:

- `dataset` = the open book.
- `dataset.width` = information about the book.
- `dataset.height` = more information about its contents.
- `dataset.read(1)` = actually reading the elevation data.

Next concept: Metadata

Try to understand these properties:

```
print(dataset.width)
print(dataset.height)
```

```
print(dataset.count)
print(dataset.dtypes)
print(dataset.crs)
print(dataset.bounds)
print(dataset.res)
print(dataset.transform)
```

These properties describe the raster without requiring you to read its entire elevation array.

Let's test your understanding.

Your MOLA dataset has:

- Width: 46,080
- Height: 23,040
- Bands: 1

What does `dataset.count` represent?

- ☐ Total number of pixels
- ☐ Number of raster bands
- ☐ Number of rows

Check answer

You:

I chose pixels. Explain whether that's correct.

You:

My answer is number of raster bands. Explain the next concept.

ChatGPT:

(Correct!) Your second answer is right: `dataset.count` represents the **number of raster bands**.

Your first answer, total number of pixels, was incorrect—but it's a useful distinction to understand.

1. What is a raster band?

Think of a raster as a collection of layers of numerical data.

For example:

Example: A three-band raster

1

Band 1

Elevation

2

Band 2

Slope

3

Band 3

Roughness

Conceptual example: three datasets aligned to the same geographical grid.

Each band contains its own grid of values.

Your MOLA dataset has only one band, containing elevation measurements.

2. Then what is the total number of pixels?

Your dataset has:

- Width = 46,080 columns
- Height = 23,040 rows
- Count = 1 band

Total pixels in one band:

$$46,080 \times 23,040 = 1,061,683,200$$

That's approximately **1.06 billion elevation cells**!

Notice that `count` is not involved in calculating the number of pixels in one band.

3. Next concept: `dataset.read(1)`

This is where things get interesting.

```
data = dataset.read(1)
```

What happens here?

Unlike reading metadata, this actually reads the pixel values from band 1 and returns a NumPy array.

For example:

```
print(data.shape)
print(data[0, 0])
```

```
print(data[9167, 32970])
```

Your earlier inspection returned approximately:

```
(23040, 46080)
```

And the elevation at your Jezero sample location was `-2645`.

One important detail: `read(1)` reads the entire band into memory by default. For your global MOLA dataset, that's roughly 2 GiB of raw int16 values, excluding additional memory overhead.

We'll learn windowed reading later to avoid that.

---

4. Quick knowledge check

Suppose a raster has:

- Width = 1000
- Height = 500
- Count = 3

How many pixels are there in each band?

- ☐ 500
- ☐ 1,500,000
- ☐ 500,000
- ☐ 3,000

Check answer

Take your time. We're building the foundation properly before returning to Marswalk routing.

You:

My answer is 500000. Explain and continue teaching me Rasterio.

ChatGPT:

(Correct!) Exactly right.

$$1000 \times 500 = 500,000$$

There are **500,000 pixels in each band**.

Since there are 3 bands, the entire raster contains 1,500,000 band-pixel values.

You're getting the distinction now!

Next concept: Reading raster data

So far, we've mostly accessed metadata. Now we're going to access the actual elevation values.

Remember:

```
dataset = rasterio.open("mola-global.tif")
```

This opens the raster and gives us a way to access it.

But it doesn't automatically load all elevation values into memory.

To read the actual values, we use:

```
data = dataset.read(1)
```

1. What does `read(1)` mean?

The number `1` identifies the band we want to read.

Rasterio uses **1-based band numbering**:

- `read(1)` → first band
- `read(2)` → second band
- `read(3)` → third band

Your MOLA dataset has one band, so `read(1)` reads its elevation data.

2. What does `data` contain?

It contains a NumPy array.

Imagine this tiny raster:

A simplified elevation array

100

105

110

115

102

108

113

118

106

111

117

122

3 rows × 4 columns = 12 elevation values.

You can access individual values using:

```
data[0, 0]
data[1, 2]
data[2, 3]
```

The indexing convention is:

```
data[row, column]
```

This is important: **row comes first, column comes second.**

3. Understanding your actual MOLA dataset

When you previously inspected your dataset, you got:

```
data.shape
```

Output:

```
(23040, 46080)
```

This means:

- 23,040 rows
- 46,080 columns

And when you ran:

```
data[9167, 32970]
```

You got:

```
-2645
```

That was the elevation value at the raster cell corresponding approximately to your Jezero sample location.

One important memory consideration: `read(1)` loads the entire band into RAM. Your global raster needs roughly 2 GiB for the raw int16 array alone.

We'll eventually learn `dataset.read(1, window=...)` to read only a small region.

Your next knowledge check

Suppose we have this array:

```
data = [
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
]
```

What will this return?

```
data[1, 2]
```

- ☐ 30
- ☐ 50
- ☐ 60
- ☐ 80

Check answer

We're going to master reading, indexing and extracting raster data before moving into coordinate transformations.

You:

My answer is 60. Check it and teach me the next Rasterio concept.

ChatGPT:

(Correct!) The answer is **60**.

Let's understand why:

```
data = [
    [10, 20, 30],  # Row 0
    [40, 50, 60],  # Row 1
    [70, 80, 90]   # Row 2
]
```

`data[1, 2]` in NumPy means:

- Row `1` → second row.
- Column `2` → third column.
- Value → `60`.

One small technical distinction: in your example, `data` is a regular Python list, so the equivalent indexing is `data[1][2]`. The comma syntax `data[1, 2]` works with NumPy arrays, not ordinary nested lists.

Next concept: Raster metadata — CRS, bounds and resolution

Now we're moving into something especially important for Marswalk.

You've already inspected these properties:

```
dataset.crs
dataset.bounds
dataset.res
dataset.transform
```

Let's understand what each one tells us.

1. CRS — Coordinate Reference System

`dataset.crs` tells us how geographical locations are represented in the raster.

For your MOLA dataset, the CRS is a Mars Equirectangular projection.

Why does this matter?

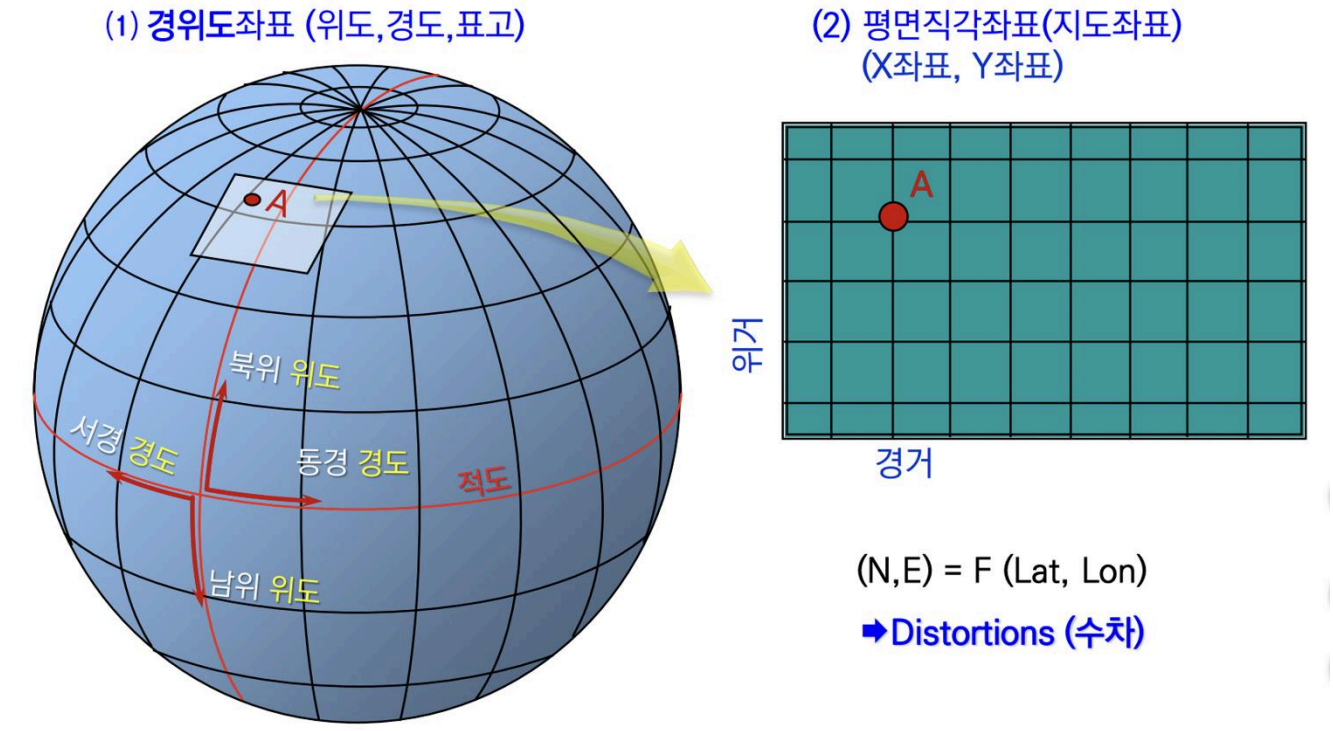
Your frontend might give us:

```
latitude = 18.38
longitude = 77.58
```

But the raster uses projected coordinates in metres.

We cannot directly use latitude and longitude as raster row and column indices.

We must transform the coordinates first.



2. Bounds — Where does the raster exist?

Your dataset returned approximately:

```
Left:   -10669675
Right:   10669675
Top:      5334838
Bottom:  -5334838
```

These values describe the outer geographical extent of the raster in its projected CRS.

Think of bounds as the four edges of your raster.

Top

Bottom

Left

Right

Raster extent

All raster cells

Conceptual illustration of the four raster bounds.

3. Resolution — How large is each pixel?

Your dataset has:

```
dataset.res
```

Output:

```
(463.0935, 463.0935)
```

This represents the pixel spacing in the raster's projected coordinate units.

For your MOLA raster, that's approximately 463 metres per pixel in both projected directions.

This is extremely important for routing.

If the rover moves across 10 horizontal pixels, the projected east-west distance is approximately:

$$10 \times 463.0935 = 4630.935 \text{ m}$$

Remember that true east-west ground distance varies with latitude in an equirectangular projection.

4. Transform — The bridge between coordinates and pixels

This is probably the most important metadata property for our next lessons.

You previously got:

```
dataset.transform
```

Output:

```
| 463.09, 0.00, -10669675.20|
| 0.00, -463.09, 5334837.60|
| 0.00, 0.00, 1.00|
```

This is an affine transformation.

It describes how pixel positions relate to projected map coordinates.

For your raster:

- X increases as columns increase.
- Y decreases as rows increase.
- The top-left corner is the origin of the raster transform.

We'll explore the mathematics behind this in the next lesson.

Knowledge check

Let's test your understanding.

Your MOLA raster has a resolution of approximately 463 metres.

What does this mean?

- ☐ Each pixel contains exactly 463 elevation units.
- ☐ Adjacent pixels are spaced approximately 463 projected metres apart.
- ☐ The entire raster is 463 metres wide.
- ☐ Each pixel represents 463 degrees of longitude.

Check answer

We're getting closer to the most exciting part: using `dataset.transform` to find the exact raster cell corresponding to a Mars coordinate.

You:

My answer is b. Check it and continue the Rasterio lesson.

ChatGPT:

(Correct!) Exactly. **B is the correct answer.**

Adjacent pixels are approximately 463.09 projected metres apart.

This tells us the spatial spacing of the raster, not the elevation stored inside each pixel.

Now we're approaching one of the most important concepts in Rasterio.

Next lesson: Understanding the Affine Transform

Remember this output from your MOLA dataset?

```
| 463.09, 0.00, -10669675.20|
| 0.00, -463.09, 5334837.60|
| 0.00, 0.00, 1.00|
```

This is called an **affine transformation**.

Its job is to connect two different coordinate systems:

- Raster coordinates: row and column.
- Map coordinates: X and Y in projected metres.

1. Understand the six important values

For a north-up raster, the transformation is:

$$X = a \times col + b \times row + c$$

$$Y = d \times col + e \times row + f$$

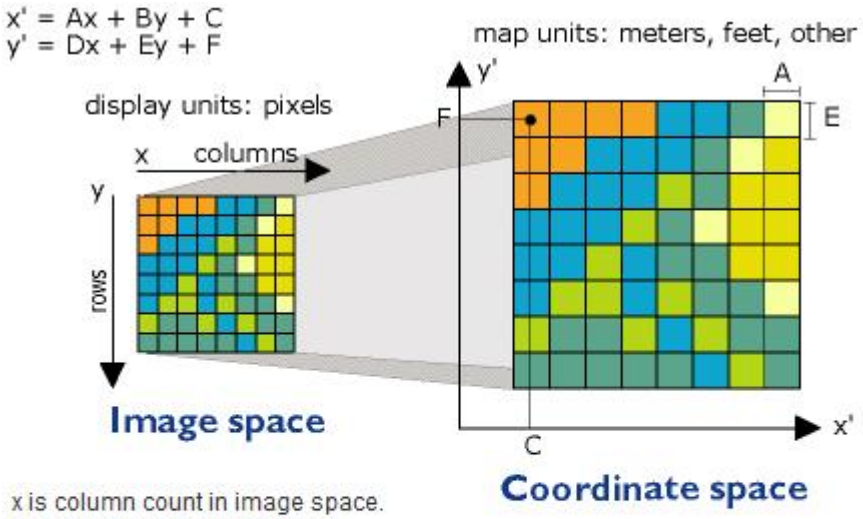
Your raster has:

Parameter	Value	Meaning
a	463.09	Pixel width
b	0	Rotation
c	-10669675.20	Left origin
d	0	Rotation
e	-463.09	Pixel height
f	5334837.60	Top origin

Notice that e is negative.

Why?

Because map Y coordinates increase northward, but raster row numbers increase downward.



x is column count in image space.
y is row count in image space.
x' is horizontal value in coodinate space.
y' is vertical value in coordinate space.

A is width of cell in map units.
B is a rotation term.
C is the x' value of the center of the upper-left cell.
D is a rotation term.
E is negative height of cell in map units.
F is the y' value of the center of the upper-left cell.

2. Let's calculate a pixel's map coordinates

Suppose we want the top-left corner of pixel:

```
row = 0
col = 0
```

Using our equations:

$$X = 463.09(0) - 10669675.20$$

$$Y = -463.09(0) + 5334837.60$$

Therefore:

```
X = -10669675.20
Y = 5334837.60
```

That's the top-left corner of the raster.

Now suppose we move one column to the right:

```
row = 0
col = 1
```

The X coordinate increases by approximately 463.09 metres.

If we move one row down:

```
row = 1
col = 0
```

The Y coordinate decreases by approximately 463.09 metres.

That's exactly how Rasterio knows where each pixel belongs geographically.

3. Pixel corner versus pixel centre

There's another detail worth understanding.

The affine transform maps pixel positions using their corners by default.

But when we want the geographical location represented by a pixel, we usually want its centre.

For pixel `(0, 0)`:

- Top-left corner: the raster origin.
- Pixel centre: 231.55 metres east and 231.55 metres south of that corner.

Rasterio provides a convenient function:

```
from rasterio.transform import xy

x, y = xy(dataset.transform, 0, 0)

print(x, y)
```

By default, `xy()` returns the pixel centre.

And to convert projected coordinates back into a row and column:

```
from rasterio.transform import rowcol

row, col = rowcol(dataset.transform, x, y)

print(row, col)
```

This is the foundation of elevation lookup in Marswalk.

---

4. Your knowledge check

Let's see whether the negative pixel height makes sense.

Your raster has:

- Pixel height: `-463.09`
- Starting Y coordinate: `5334837.60`

If we move from row 0 to row 1, what happens to Y?

- ☐ Y increases by 463.09 metres.
- ☐ Y decreases by 463.09 metres.
- ☐ Y remains unchanged.

Check answer

Our next milestone: You'll learn how to take a Mars latitude/longitude, transform it into projected X/Y, and find its exact raster row and column. That's how we'll eventually retrieve elevation at any location on Mars.

You:

My answer is: decrease. Check it and continue teaching me Rasterio.

ChatGPT:

(Correct!) Exactly. Y decreases by approximately **463.09 metres** when we move from row 0 to row 1.

Why? Because raster rows increase downward, while projected Y coordinates increase northward.

You've understood the most important part of the affine transform.

Next lesson: Converting Mars coordinates into raster positions

Now let's connect everything we've learned.

Imagine a user selects a location on the Marswalk map:

```
latitude = 18.38
longitude = 77.58
```

We want to answer:

Which pixel in our MOLA GeoTIFF contains this location?

There are three stages.

1. Geographic coordinates

18.38°N, 77.58°E

↓

2. Projected coordinates

X, Y in metres

↓

3. Raster coordinates

Row 9167, Column 32970

1. Stage one: Geographic to projected coordinates

Your MOLA CRS uses Mars Equirectangular projection.

For the CRS parameters we inspected, the simplified equations are:

$$X = R \times \lambda$$

$$Y = R \times \phi$$

Where:

- $R = 3,396,190$ metres
- λ = longitude in radians
- ϕ = latitude in radians

Remember to convert degrees into radians first.

For example:

```
import math

R = 3396190

latitude = 18.38
longitude = 77.58

lat_rad = math.radians(latitude)
lon_rad = math.radians(longitude)

x = R * lon_rad
y = R * lat_rad
```

```
print(x, y)
```

This gives us projected coordinates in metres.

This is a simplified calculation for the CRS we inspected. For production, we must verify the raster's latitude convention and use the actual CRS transformation rather than assuming every Mars dataset uses identical conventions.

2. Stage two: Projected coordinates to raster positions

Now we need to reverse the affine transformation.

For your north-up raster:

$$col = \frac{X - c}{a}$$

$$row = \frac{Y - f}{e}$$

Remember:

- $a = 463.0935$
- $c = -10669675.1973$
- $e = -463.0935$
- $f = 5334837.5987$

We divide the coordinate differences by pixel dimensions to determine the pixel position.

Rasterio already provides this calculation:

```
from rasterio.transform import rowcol

row, col = rowcol(dataset.transform, x, y)

print(row, col)
```

For your Jezero example, you previously obtained approximately:

```
9167 32970
```

3. Stage three: Retrieve the elevation

Now we can access the cell:

```
elevation = dataset.read(1)[row, col]

print(elevation)
```

Your earlier result was:

```
-2645
```

This is how a geographical location eventually becomes an elevation value.

One improvement for real Marswalk code: don't read the entire raster just to retrieve one elevation. We'll learn windowed reading and efficient dataset access soon. Also, validate bounds and NoData before returning an elevation.

---

4. Knowledge check

Let's check your understanding of the three stages.

Suppose we have:

```
latitude = 18.38
longitude = 77.58
```

We calculate:

```
x = R * lon_rad
y = R * lat_rad
```

What do x and y represent?

- ☐ Raster row and column.
- ☐ Projected map coordinates in metres.
- ☐ Elevation and slope.
- ☐ Pixel width and pixel height.

Check answer

After this lesson, we'll tackle something very important for your MOLA dataset: how to extract a small region using Rasterio windows instead of loading a 2 GB global raster into memory.

You:

My answer is b. Check it and continue teaching me Rasterio.

ChatGPT:

(Correct!) Exactly. x and y represent **projected map coordinates in metres**, not raster positions.

You're understanding the coordinate conversion process. Now let's learn something particularly important for Marswalk.

Next lesson: Windowed reading in Rasterio

Your global MOLA dataset contains:

- 46,080 columns
- 23,040 rows
- Approximately 1.06 billion cells
- About 2 GiB of raw elevation values

Imagine that a user wants to plan a route around Jezero Crater.

Do we really need to load the entire planet's elevation data into memory?

No. We can read only the geographical region we need.

That's what Rasterio's windowed reading is for.

1. What is a raster window?

A window is simply a rectangular section of a raster.

Imagine the global raster as a huge grid:

Selected

window

Small region

Global MOLA raster

Conceptual illustration: reading only the highlighted region.

Instead of reading 1.06 billion cells, we might read only 256 × 256 cells.

That's just 65,536 elevation values.

2. Understanding the Window object

Rasterio provides:

```
from rasterio.windows import Window
```

A window is created using four parameters:

```
window = Window(
    col_off,
    row_off,
    width,
    height
)
```

Let's understand them:

Parameter	Meaning
`col_off`	Starting column
`row_off`	Starting row
`width`	Number of columns to read
`height`	Number of rows to read

Notice something important: **column comes before row** in the Window constructor.

This differs from array indexing, where we use row before column.

3. Reading a window

Suppose we want a 100 × 100 section starting at row 1000 and column 2000.

```
from rasterio.windows import Window

window = Window(
    col_off=2000,
    row_off=1000,
    width=100,
    height=100
)

small_data = dataset.read(1, window=window)

print(small_data.shape)
```

Expected output:

```
(100, 100)
```

We have read only 10,000 elevation cells.

Compare:

```
# Entire band
data = dataset.read(1)

# Small region
small_data = dataset.read(1, window=window)
```

This is a fundamental technique for Marswalk.

4. Applying this to Jezero

Previously, you calculated a region around Jezero:

```
Column offset: 32842.03
Row offset:     9039.41
Width:           255.99
Height:          255.99
```

That's approximately a 256 × 256 pixel window.

You can read it using the `Window` object:

```
window = Window(
    col_off=32842,
    row_off=9039,
    width=256,
    height=256
)

jezero_data = dataset.read(1, window=window)
```

For production, we'll calculate the window from geographic bounds, round its boundaries appropriately, and verify that it remains inside the raster.

Also, a window's offsets and dimensions can be fractional. Rasterio supports this, but the returned array dimensions and pixel alignment need to be handled carefully.

5. Why this matters for Marswalk

Our eventual routing workflow will look like this:

User selects start and destination

↓

Backend calculates the geographical region

↓

Rasterio extracts a small MOLA window

↓

Calculate slope and terrain costs

↓

*A calculates the route**

This is much more efficient than loading the entire planet for every route request.

Your knowledge check

Suppose you have this window:

```
window = Window(
    col_off=500,
    row_off=200,
    width=300,
    height=150
)
```

What will be the shape of the resulting NumPy array?

- ☐ (300, 150)
- ☐ (150, 300)
- ☐ (500, 200)
- ☐ (200, 500)

Check answer

Remember: NumPy array shapes are expressed as **(rows, columns)**, even though Rasterio's Window constructor takes **(column offset, row offset, width, height)**.

You:

My answer is a. Check it and continue teaching me Rasterio.

ChatGPT:

Not quite! The correct answer is **B — `(150, 300)`**.

Don't worry; this is an important distinction, and it's easy to mix up.

1. Why is the answer `(150, 300)`?

Look at your window:

```
window = Window(
    col_off=500,
    row_off=200,
    width=300,
    height=150
)
```

The parameters mean:

- `width=300` → 300 columns.
- `height=150` → 150 rows.

But NumPy stores array shape in this order:

```
(rows, columns)
```

Therefore:

```
small_data.shape
# (150, 300)
```

Remember this rule:

- Rasterio Window: column, row, width, height.
- NumPy array: row, column.

Next lesson: Window transforms

This is especially important for Marswalk.

When you extract a small section of a global raster, the extracted array has its own local row and column positions.

For example, imagine extracting a window starting at:

```
col_off = 500
row_off = 200
```

Inside the extracted array, its top-left pixel is:

```
small_data[0, 0]
```

But in the original global raster, that same pixel is:

```
data[200, 500]
```

They represent the same geographical location!

2. Why do we need a new transform?

Your original dataset has a transform describing the entire global raster.

After extracting a window, we need a transform describing the smaller raster.

Rasterio provides:

```
window_transform = dataset.window_transform(window)
```

Now compare:

```
print(dataset.transform)
print(window_transform)
```

The new transform has a different origin because the window starts somewhere inside the global raster.

Original raster versus extracted window

Global raster

Extracted window

The extracted window gets a new local coordinate origin, while retaining its original geographical position.

3. Why this matters

Suppose you calculate slope inside your extracted Jezero region.

You might calculate a location at:

```
row = 50
col = 80
```

To convert that pixel into projected coordinates, you must use the **window transform**, not the original global transform.

Otherwise, you'll calculate the wrong geographical location.

For example:

```
from rasterio.transform import xy

x, y = xy(window_transform, 50, 80)
```

This gives the projected coordinates of the centre of the corresponding pixel in the original Mars dataset.

That's how we preserve geographical accuracy after extracting a region.

---

4. Knowledge check

Suppose:

```
window = Window(
    col_off=1000,
    row_off=500,
    width=200,
    height=100
)
```

The extracted array's pixel is:

```
small_data[20, 30]
```

Which pixel does it correspond to in the original global raster?

- ☐ `data[20, 30]`
- ☐ `data[30, 20]`
- ☐

`data[520, 1030]`

- [ ]

`data[1030, 520]`

Check answer

After this, we'll learn **NoData values and raster masks**—another essential topic before we can safely extract elevation values for Marswalk.

You:

My answer is a. Check it and continue teaching me Rasterio.

ChatGPT:

Not quite! The correct answer is **C —** `data[520, 1030]`.

Let's work through it.

Your extracted window starts at:

```
col_off = 1000
row_off = 500
```

You selected a pixel inside that window:

```
small_data[20, 30]
```

To find its original position, add the offsets:

$$GlobalRow = 500 + 20 = 520$$

$$GlobalCol = 1000 + 30 = 1030$$

Therefore:

```
data[520, 1030]
```

Remember: Local position + window offset = Global position.

Next lesson: NoData values and raster masks

This is one of the most important topics for Marswalk.

Not every pixel in a raster necessarily contains a valid elevation measurement.

Some pixels represent missing data.

1. What is NoData?

Your MOLA dataset has this property:

```
dataset.nodata
```

Your earlier output was:

```
-32768.0
```

This means `-32768` is the designated NoData value for your dataset.

Imagine this small raster:

Example elevation grid

100

120

130

140

110

NoData

135

145

115

125

140

NoData

Valid elevation · Missing elevation

Notice that `-32768` is not an actual elevation we should use in our calculations.

Imagine calculating slope between:

- Cell A: -2000 metres
- Cell B: -32768 (NoData)

The result would be meaningless.

That's why we must identify invalid pixels before terrain analysis.

2. How Rasterio identifies NoData

You can inspect it:

```
print(dataset.nodata)
```

You can also read a mask:

```
mask = dataset.read_masks(1)
```

Rasterio returns an array of mask values.

Generally:

- `0` = invalid pixel.
- `255` = valid pixel.

For example:

```
print(mask[100, 200])
```

This tells you whether that pixel is considered valid.

3. Another useful method: masked arrays

Rasterio can automatically mask NoData values:

```
data = dataset.read(1, masked=True)
```

This returns a NumPy masked array.

Unlike an ordinary array, it keeps track of which values should be excluded from calculations.

For example:

```
print(data[0, 0])
```

```
print(data.mask[0, 0])
```

A masked value won't accidentally be treated as an ordinary elevation.

One detail: Rasterio masks can also reflect internal dataset masks, not just the NoData number. That's why checking `read_masks()` is useful.

4. Why this matters for Marswalk

Imagine A* explores a cell where MOLA has no valid elevation.

What should happen?

We cannot calculate its slope reliably from missing elevation data.

For a safety-first rover planner, we should initially treat unknown terrain as **unverified and unavailable for routing**, rather than assuming it's safe.

Later, we can introduce other datasets or explicit uncertainty handling.

Knowledge check

Your MOLA dataset has:

```
dataset.nodata = -32768
```

Suppose a cell contains:

```
elevation = -32768
```

Should Marswalk calculate its slope using this value?

- ☐ Yes, because it is a numerical elevation value.
- ☐ No, because it represents missing data.
- ☐ Only if the neighbouring elevations are valid.

Check answer

After this, we'll learn how to build a proper elevation lookup function that takes Mars latitude and longitude and returns a valid elevation—or tells us when no valid data exists.

You:

My answer is yes. Check it and continue teaching me Rasterio.

ChatGPT:

Not quite! The correct answer is **No — `-32768` represents missing data, not a real elevation measurement.**

This is a very important distinction in geospatial programming.

1. Why can't we use NoData?

Imagine two neighbouring MOLA cells:

```
elevation_a = -2500
elevation_b = -32768
```

If we calculate the difference:

$$-32768 - (-2500) = -30268$$

We get a difference of 30,268 metres!

But that's completely misleading. The second cell has no valid elevation measurement.

For Marswalk, using this value could produce:

- Incorrect slope calculations.
- Incorrect terrain costs.
- Misleading route decisions.

Our rule: Never treat NoData as an actual elevation.

2. Understanding masked arrays

Let's revisit:

```
data = dataset.read(1, masked=True)
```

The `masked=True` argument tells Rasterio to return a masked array.

A masked array has two important parts:

- The numerical values.
- A mask identifying which values are invalid.

For example, conceptually:

Elevation values

100

120

130

140

110

--

135

145

115

125

140

--

Corresponding validity mask

Valid

Valid

Valid

Valid

Valid

Invalid

Valid

Valid

Valid

Valid

Valid

Invalid

Conceptual example: 1 represents valid, 0 represents invalid.

You can inspect the mask with:

```
mask = dataset.read_masks(1)
```

Remember:

- `255` = valid.
- `0` = invalid.

You can also let Rasterio handle masking:

```
data = dataset.read(1, masked=True)
```

This is generally preferable to manually checking only whether a value equals `-32768`, because raster validity can also be represented by internal masks.

3. Next concept: Building an elevation lookup

Now we're ready to connect the concepts we've learned.

Eventually, Marswalk needs a function that works like this:

```
elevation = get_elevation(
    latitude=18.38,
    longitude=77.58
)
```

It should return either:

- A valid elevation value.
- `None` if the location is outside the raster or has invalid data.

The process:

1. **Receive latitude and longitude** Example: 18.38°N, 77.58°E
2. **Convert into projected X/Y** Use the raster's CRS.
3. **Find row and column** Use `dataset.index(x, y)` or Rasterio's `rowcol()`.
4. **Check raster boundaries** Ensure the row and column exist.
5. **Read the pixel** Read only the required cell or a small window.
6. **Check validity** Reject masked or NoData values.
7. **Return elevation** Return a valid value or `None`.

Notice how all our previous lessons connect to this one function.

4. A first look at the Rasterio API

You already know:

```
from rasterio.transform import rowcol
```

```
row, col = rowcol(dataset.transform, x, y)
```

Rasterio also offers a convenient shortcut:

```
row, col = dataset.index(x, y)
```

Both help convert projected coordinates into raster indices.

For a single elevation lookup, we can read just one cell using a one-pixel window rather than loading the entire raster.

---

5. Knowledge check

Suppose we have:

```
row, col = dataset.index(x, y)
```

The resulting position is:

```
row = 25000
col = 100
```

But your raster has only 23,040 rows.

What should our elevation lookup function do?

- ☐ Read the pixel anyway.
- ☐ Return None because the position is outside the raster.
- ☐ Change the row to 23,039.
- ☐ Return the NoData value.

Check answer

We're almost finished with the fundamental Rasterio concepts. After this, I'd like us to build the elevation lookup function together, one piece at a time, rather than just handing you the finished code.

You:

My answer is b. Check it and continue teaching me Rasterio.

ChatGPT:

(Correct!) Exactly. We return `None` because row 25,000 is outside the raster.

Your MOLA raster has valid row indices from `0` to `23,039` and column indices from `0` to `46,079`.

We should never silently change an invalid coordinate to the nearest edge pixel. That would return elevation for a different location.

Next lesson: Reading one pixel efficiently

We've learned how to read an entire raster and how to extract a window. Now let's learn how to read just **one elevation cell**.

This is useful because Marswalk may need to retrieve elevations for many individual coordinates.

1. The one-pixel window

Remember this?

```
from rasterio.windows import Window
```

We can create a window containing exactly one pixel:

```
window = Window(
    col_off=32970,
    row_off=9167,
    width=1,
    height=1
)
```

Then read it:

```
elevation = dataset.read(
    1,
    window=window,
    masked=True
)

print(elevation)
```

The resulting array has this shape:

```
(1, 1)
```

Why? Because we requested one row and one column.

To retrieve the actual value:

```
value = elevation[0, 0]
```

Notice that the indices are now local to the small window, not the global raster.

2. Let's make the operation reusable

Imagine we already know the correct row and column:

```
row = 9167
col = 32970
```

We could create a small function:

```
from rasterio.windows import Window

def read_pixel(dataset, row, col):

    window = Window(
        col_off=col,
        row_off=row,
        width=1,
        height=1
    )

    value = dataset.read(
        1,
        window=window,
        masked=True
    )[0, 0]

    if value is ...:
        return None

    return value
```

I've deliberately left one part incomplete. We'll fill it in together.

The missing condition needs to identify whether the returned value is masked.

For a NumPy masked array, the relevant property is:

```
value is np.ma.masked
```

However, since you're learning the complete Rasterio workflow, we'll also need to check the raster boundaries before reading.

3. A useful alternative: `dataset.sample()`

Rasterio has another method:

```
samples = dataset.sample([(x, y)])
```

It samples raster values at projected coordinates.

This can be convenient when retrieving many point elevations.

But remember:

- `x` and `y` must be in the raster's CRS.
- Check the sample's validity.
- It does not convert latitude/longitude into the raster CRS for you.
- It doesn't replace proper bounds and NoData handling.

We'll compare both approaches when we implement the actual lookup function.

4. One more important detail: elevation units

Your MOLA raster uses `int16`, but that alone doesn't establish the physical unit or any scale/offset applied to the stored values.

When we return to the actual dataset, inspect:

```
print(dataset.scales)
print(dataset.offsets)
print(dataset.units)
```

We must also verify the USGS product documentation.

We don't want Marswalk reporting an elevation in metres if the stored values require a conversion.

Knowledge check

Suppose we read one pixel:

```
elevation = dataset.read(
    1,
    window=Window(
        col_off=100,
        row_off=200,
        width=1,
        height=1
    ),
    masked=True
)
```

What is the shape of `elevation`?

- ☐ (200, 100)
- ☐ (100, 200)
- ☐ (1, 1)
- ☐ (1,)

Check answer

Our next milestone: Build `get_elevation(latitude, longitude)` together, including coordinate conversion, bounds checking, windowed reading and NoData handling.

You:

My answer is c. Check it and continue teaching me Rasterio.

ChatGPT:

(Correct!) The answer is **C — `(1, 1)`**.

You requested:

- Width = 1 column.
- Height = 1 row.

NumPy represents the result as `(rows, columns)`, so the shape is `(1, 1)`.

You're now familiar with the main Rasterio operations: opening datasets, reading bands, indexing, transforms, windows and masks.

Let's put them together.

Next lesson: Building `get_elevation()`

This is going to be one of the most useful functions in Marswalk.

Our goal is to eventually write:

```
elevation = get_elevation(
    latitude=18.38,
    longitude=77.58
)
```

And receive a valid elevation or `None`.

1. Understand the function's responsibilities

1. Receive latitude and longitude.
2. Convert geographic coordinates into the raster's CRS.
3. Find the corresponding row and column.
4. Check whether the pixel is inside the raster.
5. Read the elevation using a one-pixel window.
6. Check whether the value is valid.
7. Return the elevation.

We'll implement these one at a time.

2. First: Validate the raster position

Suppose Rasterio gives us:

```
row = 9167
col = 32970
```

Your dataset has:

```
dataset.height  # 23040
dataset.width   # 46080
```

A valid position must satisfy:

$$0 \leq row < height$$

$$0 \leq col < width$$

In Python:

```
if not (0 <= row < dataset.height):
    return None
```

```
    if not (0 <= col < dataset.width):
        return None
```

Why do we check this?

Because attempting to read a pixel outside the raster is invalid. More importantly, we shouldn't accidentally return an elevation for a different location.

3. Second: Read and validate the pixel

We can read a single pixel:

```
window = Window(
    col_off=col,
    row_off=row,
    width=1,
    height=1
)

value = dataset.read(
    1,
    window=window,
    masked=True
)[0, 0]
```

Now comes the important part.

Because we used `masked=True`, we can check:

```
np.ma.is_masked(value)
```

If it returns `True`, the pixel is invalid.

Otherwise, we can use its value.

One correction to the earlier example: `value is ...` was just a placeholder, not a real NoData check. `np.ma.is_masked()` is the appropriate check here.

---

4. Something important about Mars coordinates

I want to highlight a potential issue before we implement the full function.

Your MOLA raster uses a Mars-specific CRS.

We must not blindly use Earth's `EPSG:4326` to transform Mars latitude/longitude. We need a geographic CRS compatible with the Mars dataset's datum and longitude convention.

For the real implementation, we'll inspect the CRS definition and use an appropriate transformation. Our earlier spherical equations are useful for understanding the mathematics, but we must validate them against the dataset.

Your exercise

Before we write the complete function, I want you to implement just the boundary validation.

Create a function called:

```
def is_inside_raster(dataset, row, col):
```

It should return:

- `True` if the row and column are valid.
- `False` otherwise.

Test these three positions using your MOLA dimensions:

Row	Column	Expected
9167	32970	True
25000	100	False
100	50000	False

Don't copy a complete implementation from me. Try writing it yourself. Send me your code, and I'll review it.

Once this works, we'll connect coordinate conversion, windowed reading and NoData handling into your first real Marswalk elevation lookup function.

You:

dataset = rasterio.open("mola.tif")

def is_inside_raster(dataset, row, col): if(row > dataset.height or col > dataset.width): return False else: return True

print(is_inside_raster(dataset, 9167, 32970)) print(is_inside_raster(dataset, 25000, 100)) print(is_inside_raster(dataset, 100, 50000))

ChatGPT:

Nice! Your logic is almost correct, and your three test cases will actually produce the expected results.

But there's an important bug hiding in your function. Let's review it.

1. What you did correctly

```
if(row > dataset.height or col > dataset.width):
    return False
else:
    return True
```

You correctly understood that:

- Row must not exceed the raster height.
- Column must not exceed the raster width.
- If either is outside, return `False`.

2. The hidden bug

Your condition uses `>` instead of `>=`.

Remember, your raster has:

```
dataset.height = 23040
dataset.width = 46080
```

Valid indices are:

Dimension	Valid indices
Rows	0–23039
Columns	0–46079

What happens if someone passes:

```
is_inside_raster(dataset, 23040, 100)
```

Your function returns `True`!

But row `23040` doesn't exist.

There's another issue: negative indices should also be rejected. Python normally allows negative indexing, but a negative raster row or column isn't a valid geographic pixel position.

3. Your correction

Try expressing the valid condition directly:

```
0 <= row < dataset.height
0 <= col < dataset.width
```

These are called chained comparisons in Python.

Your task is to rewrite the function so that it returns `True` only when **both** conditions are satisfied.

4. Add these test cases

Boundary testing checklist

Valid position

`{t.code}` · (True)

Row too large

`{t.code}` · (False)

Column too large

`{t.code}` · (False)

Negative row

`{t.code}` · (False)

Exact height boundary

`{t.code}` · (False)

Exact width boundary

`{t.code}` · (False)

One more small programming habit: eventually, we'll use a `with rasterio.open(...) as dataset:` block to ensure the file is properly closed when we're finished.

Send me your corrected function. After that, we'll move to the actual elevation lookup function.

You:

dataset = rasterio.open("mola.tif")

def is_inside_raster(dataset, row, col): if((row < 0 or row >= dataset.height) or (col < 0 or col >= dataset.width)): return False else: return True

print(is_inside_raster(dataset, 9167, 32970)) print(is_inside_raster(dataset, 25000, 100)) print(is_inside_raster(dataset, 100, 50000))

ChatGPT:

Excellent! (All boundary conditions handled correctly)

Your updated function is logically correct.

Let's inspect it:

```
def is_inside_raster(dataset, row, col):
    if ((row < 0 or row >= dataset.height)
        or (col < 0 or col >= dataset.width)):
        return False
    else:
        return True
```

Why it works

You check two things:

1. Is the row outside the valid range?
2. Is the column outside the valid range?

If either is true, the position is invalid.

Your three original test cases should produce:

```
True
False
False
```

And your function also correctly rejects negative indices and exact upper boundaries.

A small Python improvement

You can simplify the function:

```
def is_inside_raster(dataset, row, col):
    return (
        0 <= row < dataset.height
        and 0 <= col < dataset.width
    )
```

Both versions are correct. The second is more concise.

Next step: Coordinate to pixel conversion

Now we're ready to connect two things you've learned:

- Coordinate transformations.
- Raster boundary validation.

We'll create a function that accepts projected coordinates and returns a valid raster position.

First, let's understand one Rasterio method:

```
row, col = dataset.index(x, y)
```

Here:

- `x` = projected X coordinate.
- `y` = projected Y coordinate.
- `row` = resulting raster row.
- `col` = resulting raster column.

Notice that `dataset.index()` does the pixel-index calculation using the dataset's georeferencing.

Your next exercise

Write a function:

```
def get_pixel_position(dataset, x, y):
```

It should:

1. Convert X/Y into row/column using `dataset.index()`.
2. Call your `is_inside_raster()` function.
3. Return `(row, col)` if valid.
4. Return `None` if invalid.

For now, don't worry about converting latitude/longitude. We'll add that separately.

Why separate these functions?

Because each function should have one responsibility:

- `is_inside_raster()` checks boundaries.
- `get_pixel_position()` converts projected coordinates into pixel indices.
- Later, `get_elevation()` will combine the necessary steps.

That's how we'll keep the Marswalk backend maintainable.

Send me your implementation when you're ready.

You:

dataset = rasterio.open("mola.tif")

def is_inside_raster(dataset, row, col): if((row < 0 or row >= dataset.height) or (col < 0 or col >= dataset.width)): return False else: return True

def get_pixel_position(dataset, x, y): if(is_inside_raster(dataset, x, y)): row, col = rowcol(transform=dataset.transform, x=x, y=y) return (row, col)

```
return None
```

ChatGPT:

You're getting the structure right! I like that you reused `is_inside_raster()` instead of duplicating its logic.

However, there's **one important logical mistake** in your implementation. Let's find it.

1. The problem

You wrote:

```
def get_pixel_position(dataset, x, y):
    if(is_inside_raster(dataset, x, y)):
        row, col = rowcol(transform=dataset.transform, x=x, y=y)
        return (row, col)

    return None
```

Look carefully at this line:

```
is_inside_raster(dataset, x, y)
```

What does `is_inside_raster()` expect?

```
is_inside_raster(dataset, row, col)
```

But you're giving it projected coordinates!

Remember the units:

Value	Meaning	Unit
x	Projected X	Metres
y	Projected Y	Metres

Value	Meaning	Unit
`row`	Raster row	Pixel index
`col`	Raster column	Pixel index

You cannot check raster boundaries using projected coordinates directly.

2. Correct order of operations

We must first convert X/Y into row/column.

Then check whether those indices are valid.

Input: X, Y

Projected coordinates in metres

↓

dataset.index(x, y)

Convert to row, column

↓

is_inside_raster(dataset, row, col)

Check boundaries

↓

(Return (row, col) or None)

3. Your correction task

Rasterio provides a convenient method:

```
row, col = dataset.index(x, y)
```

I want you to change your function so that:

1. Convert X/Y first.
2. Check the resulting row/column.
3. Return the tuple if valid.
4. Otherwise return `None`.

You can keep your existing `is_inside_raster()` function unchanged.

One additional thing: `dataset.index()` is a convenient alternative to manually calling `rowcol()` with the affine transform.

---

4. Test cases

Once corrected, test:

- A projected coordinate corresponding to Jezero.
- A coordinate outside the raster.
- A coordinate near the raster boundary.

For the Jezero test, we previously obtained approximately row `9167`, column `32970`.

Send me your revised code. Then we'll move on to reading the actual elevation value from the returned pixel position.

You're learning an important backend design principle here: **the order of data transformations matters.**

You:

dataset = rasterio.open("mola.tif")

def is_inside_raster(dataset, row, col): if((row < 0 or row >= dataset.height) or (col < 0 or col >= dataset.width)): return False else: return True

def get_pixel_position(dataset, x, y): row, col = dataset.index(x,y) if(is_inside_raster(dataset, row, col)): return (row, col)

```
return None
```

ChatGPT:

Perfect! This time your logic is correct. (Approved)

You've fixed the order of operations:

1. Receive projected coordinates.
2. Convert them into raster indices.
3. Validate the indices.
4. Return the position or `None`.

Your function:

```
def get_pixel_position(dataset, x, y):
    row, col = dataset.index(x, y)

    if is_inside_raster(dataset, row, col):
        return (row, col)

    return None
```

That's exactly the logic we want.

One improvement to understand

You might wonder why we don't just check the projected coordinates against `dataset.bounds`.

We actually can! But checking the resulting row and column is useful because it validates the exact pixel indices we're about to access.

Also, `dataset.index()` normally uses the containing pixel convention (floor for a north-up raster). A coordinate exactly on the outer right or bottom edge can produce an out-of-range index, which our validation catches.

Next lesson: Reading elevation from a pixel

Now we're going to combine what you've learned.

We have:

```
position = get_pixel_position(dataset, x, y)
```

Suppose it returns:

```
(9167, 32970)
```

We now need to read the elevation at that position.

1. Reading one pixel with a window

Remember our earlier lesson?

```
from rasterio.windows import Window

window = Window(
    col_off=32970,
    row_off=9167,
    width=1,
    height=1
)
```

Then:

```
value = dataset.read(
    1,
    window=window,
    masked=True
)
```

The result is a 1×1 masked array.

To access the actual cell:

```
elevation = value[0, 0]
```

2. Handling invalid data

Now we must determine whether the value is masked.

```
import numpy as np

if np.ma.is_masked(elevation):
    return None
```

If it's not masked, we can return its numerical value.

One more consideration: check that your dataset has the correct units and scale/offset before interpreting that number as metres.

3. Your next task

Create a function:

```
def get_elevation_from_pixel(dataset, row, col):
```

Requirements:

1. Validate the row and column using `is_inside_raster()`.
2. Create a 1×1 `Window`.
3. Read band 1 using `masked=True`.
4. Extract the value at `[0, 0]`.
5. Return `None` if the value is masked.
6. Otherwise return the elevation.

You already know every concept required to build this function.

Don't rush. Write it yourself, and I'll review it.

After that, we'll combine it with `get_pixel_position()` to build your first complete elevation lookup pipeline for Marswalk.

You:

def get_elevation_from_pixel(dataset, row, col): if(is_inside_raster(dataset, row, col)): window = Window(col_off=32970, row_off=9167, width=1, height=1)

```
    data = dataset.read(1, window=window, masked=True)
    elevation = data[0, 0]

    if(np.ma.is_masked(elevation)):
        return None

    return elevation

return None
```

ChatGPT:

Very close! Your function has the correct overall structure, and you've correctly handled boundary validation and masked values.

But there's **one important bug** that we need to fix.

1. The problem: Hardcoded coordinates

You wrote:

```
window = Window(
    col_off=32970,
    row_off=9167,
    width=1,
    height=1
)
```

Remember, your function receives:

```
def get_elevation_from_pixel(dataset, row, col):
```

So why are we always reading column `32970` and row `9167`?

Those are the Jezero coordinates from our earlier example. We want this function to work for *any* pixel.

The window should use the function arguments:

- `col_off` should use `col`.
- `row_off` should use `row`.

That's the only logical correction needed.

2. Let's review what you did right

Requirement	Your implementation
Validate pixel position	(Correct)
Create 1×1 window	(Correct)
Read band 1	(Correct)
Use `masked=True`	(Correct)
Detect masked value	(Correct)
Return elevation	(Correct)

Also, remember to import:

```
import numpy as np
from rasterio.windows import Window
```

3. Your correction task

Fix the hardcoded window offsets so that your function reads whichever pixel is passed to it.

Then test:

```
print(get_elevation_from_pixel(dataset, 9167, 32970))
print(get_elevation_from_pixel(dataset, 9168, 32970))
print(get_elevation_from_pixel(dataset, 25000, 100))
```

The first should reproduce your earlier Jezero sample value of approximately `-2645`, assuming the same dataset and indexing convention.

What you've accomplished

You're now building reusable Rasterio functions rather than just running inspection commands.

We've developed:

1. `is_inside_raster()` — validates pixel positions.
2. `get_pixel_position()` — converts projected coordinates into pixel indices.

3. `get_elevation_from_pixel()` — retrieves a valid elevation value.

Next, we'll connect these three functions into one:

`get_elevation(latitude, longitude)`

That will be our first complete building block for Marswalk's terrain-processing backend.

You:

Ok

ChatGPT:

Great. Fix the hardcoded window offsets so the function uses its arguments:

```
window = Window(
    col_off=col,
    row_off=row,
    width=1,
    height=1
)
```

Then run the three tests from the previous step. Share your updated function and the outputs, and we'll review them before moving on to the next step.

You:

dataset = rasterio.open("mola.tif")

def is_inside_raster(dataset, row, col): if((row < 0 or row >= dataset.height) or (col < 0 or col >= dataset.width)): return False else: return True

def get_pixel_position(dataset, x, y): row, col = dataset.index(x,y) if(is_inside_raster(dataset, row, col)): return (row, col)

```
return None
```

def get_elevation_from_pixel(dataset, row, col): if(is_inside_raster(dataset, row, col)): window = Window(col_off=col, row_off=row, width=1, height=1)

```
    data = dataset.read(1, window=window, masked=True)
    elevation = data[0, 0]

    if(np.ma.is_masked(elevation)):
        return None
```

```
    return elevation

return None
```

ChatGPT:

Nice work! Your functions are now logically correct. You've fixed the hardcoded coordinates, and `get_elevation_from_pixel()` now reads the pixel you actually request.

Let's review each function:

1. **`is_inside_raster()`** — checks that the row and column are within the raster dimensions.
2. **`get_pixel_position()`** — converts projected coordinates `(x, y)` into `(row, col)`, then checks whether that pixel exists.
3. **`get_elevation_from_pixel()`** — reads exactly one pixel using a `1 × 1` window and returns `None` if the pixel is NoData or outside the raster.

One small improvement

You can simplify the first function:

```
def is_inside_raster(dataset, row, col):
    return 0 <= row < dataset.height and 0 <= col < dataset.width
```

Your original version is also correct; this is just more concise.

Next step: test your functions

Try these:

```
print(get_pixel_position(dataset, 0, 0))
print(get_elevation_from_pixel(dataset, 9167, 32970))
print(get_elevation_from_pixel(dataset, 9168, 32970))
print(get_elevation_from_pixel(dataset, -1, 100))
```

The second result should be around `-2645` if you're using the same dataset.

One important question: What does `get_elevation_from_pixel()` return when the pixel is outside the raster, and why?

Once you've answered and tested it, we'll combine these functions into `get_elevation()` so you can request an elevation using coordinates rather than manually supplying row and column.

You:

(11520, 23040) -2645 -2651 None because out of bound

ChatGPT:

Excellent! All four results are correct. You've now verified the complete pixel-to-elevation workflow.

Let's understand your first result:

```
get_pixel_position(dataset, 0, 0)
# (11520, 23040)
```

That's the raster pixel corresponding to projected coordinates `(x=0, y=0)`. It makes sense because the raster is centred around the Mars equator and prime meridian.

And your last answer is correct: an out-of-bounds pixel returns `None` because `is_inside_raster()` rejects it before Rasterio tries to read it.

Next step: combine the functions

Now let's create a higher-level function:

```
def get_elevation(dataset, x, y):
    # Your implementation
```

Its job is simple:

1. Receive projected coordinates `(x, y)`.
2. Find the corresponding `(row, col)`.
3. Check whether the pixel is inside the raster.
4. Retrieve its elevation.
5. Return the elevation, or `None` if unavailable.

Your exercise: Implement this function using the two functions you've already written. Don't copy their internal logic again—reuse them.

One important distinction: these inputs are projected coordinates in metres, not latitude and longitude yet. We'll handle that conversion separately.

You:

dataset = rasterio.open("mola.tif")

def is_inside_raster(dataset, row, col): if((row < 0 or row >= dataset.height) or (col < 0 or col >= dataset.width)): return False else: return True

def get_pixel_position(dataset, x, y): row, col = dataset.index(x,y) if(is_inside_raster(dataset, row, col)): return (row, col)

```
return None
```

def get_elevation_from_pixel(dataset, row, col): if(is_inside_raster(dataset, row, col)): window = Window(col_off=col, row_off=row, width=1, height=1)

```
    data = dataset.read(1, window=window, masked=True)
    elevation = data[0, 0]

    if(np.ma.is_masked(elevation)):
        return None

    return elevation

return None
```

def get_elevation(dataset, x, y): row, col = get_pixel_position(dataset, x, y) elevation = get_elevation_from_pixel(dataset, row, col) return elevation

---

ChatGPT:

You're very close! The function correctly reuses your existing functions. There's just **one edge case** to handle.

The problem

Your `get_pixel_position()` returns either:

- `(row, col)` when the coordinates are inside the raster.
- `None` when they're outside.

But this line assumes it always returns a tuple:

```
row, col = get_pixel_position(dataset, x, y)
```

If it returns `None`, Python cannot unpack it into `row` and `col`. You'll get a `TypeError`.

Your correction

Check the result before unpacking it:

```
def get_elevation(dataset, x, y):
    position = get_pixel_position(dataset, x, y)

    if position is None:
        return None
```

```
    row, col = position
    elevation = get_elevation_from_pixel(dataset, row, col)

    return elevation
```

Test your function

```
print(get_elevation(dataset, 0, 0))
print(get_elevation(dataset, -20000000, 0))
```

The second call should return `None` because the coordinates are outside the raster.

What you've built: A reusable pipeline that converts projected coordinates into a raster pixel and retrieves its elevation without loading the entire MOLA dataset into memory.

Next, we'll make this function accept **Mars latitude and longitude** instead of projected X/Y coordinates.

You:

-1517 None

ChatGPT:

Perfect! (Both tests passed)

- `(0, 0)` → `-1517` metres (assuming the dataset's elevation values are in metres).
- `(-20000000, 0)` → `None`, correctly identifying coordinates outside the raster.

You've now built a working **coordinate → pixel → elevation** pipeline.

Next lesson: Latitude and longitude

Currently, your function accepts projected coordinates in metres:

```
get_elevation(dataset, x, y)
```

But users will select locations on the Mars map using latitude and longitude.

Our next goal is:

```
get_elevation_from_latlon(dataset, latitude, longitude)
```

Before writing it, let's check your understanding.

For a spherical Mars model, with radius

$$R$$

, the simplified equirectangular conversion is:

$$x = R \times \text{radians}(\text{longitude})$$

$$y = R \times \text{radians}(\text{latitude})$$

Question: Why do we convert degrees into radians before multiplying by the radius?

You:

No idea

ChatGPT:

No problem! Let's build the understanding from the beginning.

1. What is a radian?

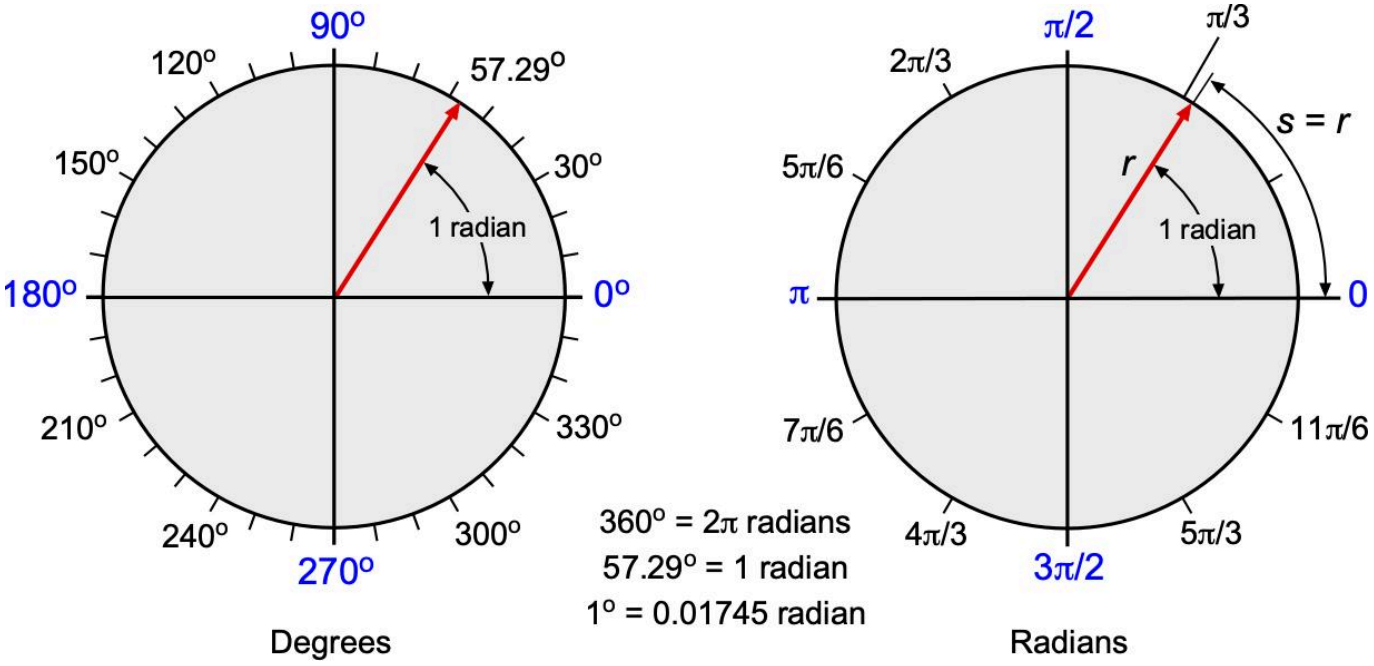
There are two common ways to measure angles:

- **Degrees:** A full circle is 360°.
- **Radians:** A full circle is

  $$2\pi$$

  radians (approximately 6.283).

They measure the same thing, just using different units.



Some useful conversions:

Degrees	Radians
0°	0
90°	π/2
180°	π
270°	3π/2
360°	2π

2. Why do we need radians?

Imagine Mars as a sphere.

If you travel along its equator, the distance around it is:

$$C = 2\pi R$$

The important relationship is:

$$\text{Distance} = R \times \text{Angle in radians}$$

That's why radians are useful: **multiplying a radius by an angle in radians gives the distance along a circle's arc.**

For longitude, this gives our simplified Mars coordinate conversion:

$$x = R \times \text{radians}(\text{longitude})$$

We use the same idea for latitude in the equirectangular projection.

3. Let's try a small exercise

Suppose Mars has a radius of 3,396,190 metres.

What is the radian equivalent of 180 degrees?

Choose your answer:

- ☐ π radians
- ☐ 2π radians
- ☐ π/2 radians
- ☐ 180 radians

Check answer

Take your time. We'll understand the geometry before writing more code.

You:

180 degrees equals pi radians.

ChatGPT:

Correct! (That's right)

$$180^\circ = \pi \text{ radians}$$

Now let's connect this to Python.

Python's `math` module provides a function that converts degrees to radians:

```
import math

angle = math.radians(180)

print(angle)  # 3.141592653589793
```

Your next exercise

What do you think this code will print?

```
import math

print(math.radians(90))
```

Will it print:

- `1.5708`
- `3.14159`
- `6.28318`

Once you've got this, we'll calculate the projected X coordinate for a Mars location.

You:

1. `1.5708`
2.

ChatGPT:

Exactly! **90° = π/2 radians ≈ 1.5708.**

Now let's apply this to Mars.

Calculate projected X

Suppose a location has longitude **90° East**.

Mars radius:

$$R = 3,396,190 \text{ metres}$$

Our formula:

$$x = R \times \text{radians}(\text{longitude})$$

In Python:

```
import math

mars_radius = 3396190
longitude = 90

x = mars_radius * math.radians(longitude)

print(x)
```

The result should be approximately:

$$x = 5,334,838 \text{ metres}$$

Your turn

What would the projected X coordinate be if longitude were **0°**?

Think about what happens when you convert zero degrees into radians and multiply by the Mars radius.

You:

0

ChatGPT:

Correct!

$$x = 3,396,190 \times \text{radians}(0)$$

$$x = 3,396,190 \times 0 = 0$$

So longitude 0° gives projected X = **0 metres**.

Now let's calculate both coordinates

For our simplified equirectangular Mars projection:

$$x = R \times \text{radians}(\text{longitude})$$

$$y = R \times \text{radians}(\text{latitude})$$

Let's test your understanding.

A location has:

- Latitude: 30° North
- Longitude: 60° East

Question: Which coordinate represents latitude, X or Y?

A) X

B) Y

You:

X

ChatGPT:

Almost! The correct answer is **B) Y**.

Remember:

- **Longitude → X** (east-west position)
- **Latitude → Y** (north-south position)

For our example:

- Latitude = 30° → calculates Y.
- Longitude = 60° → calculates X.

Think of a standard map:

- X moves horizontally.
- Y moves vertically.

Quick memory check

If a Mars location is at:

- Latitude = 0°
- Longitude = 45° East

Which projected coordinate will be zero?

A) X

B) Y

C) Both

You:

X

ChatGPT:

Not quite! The correct answer is **B) Y**.

Let's calculate:

$$Y = R \times \text{radians}(0^\circ) = 0$$

$$X = R \times \text{radians}(45^\circ)$$

So:

- X is positive because longitude is 45° East.
- Y is zero because latitude is 0° (the equator).

Remember: Latitude controls Y; longitude controls X.

One last check: If latitude is **-30°** (30° South), will Y be positive or negative?

You:

negative

ChatGPT:

Exactly! (Correct)

A southern latitude is negative, so:

$$Y = R \times \text{radians}(-30^\circ)$$

Therefore, Y is negative.

You've now understood the basic relationship:

Geographic coordinate	Projected coordinate
Longitude East	Positive X
Longitude West	Negative X
Latitude North	Positive Y
Latitude South	Negative Y

Next exercise: Write the conversion function

Let's implement what you've learned.

Complete this function yourself:

```
import math

def mars_latlon_to_xy(latitude, longitude):
    R = 3396190

    x = # longitude conversion
    y = # latitude conversion

    return x, y
```

Test it with:

```
x, y = mars_latlon_to_xy(18.38, 77.58)

print(x, y)
```

Send me your implementation and output. We'll then connect it to your existing `get_elevation()` function.

Note: This is the simplified spherical equirectangular conversion matching the MOLA projection parameters we've inspected. We'll verify the CRS details before treating it as production-ready.

You:

def mars_latlon_to_xy(latitude, longitude): R = 3396190

```
x = R * longitude *  math.pi / 180
y = R * latitude *  math.pi / 180

return x, y
```

x, y = mars_latlon_to_xy(18.38, 77.58)

print(x, y)

ChatGPT:

Excellent! Your implementation is mathematically correct. You've successfully converted Mars latitude and longitude into projected X/Y coordinates.

Your code:

```
x = R * longitude * math.pi / 180
y = R * latitude * math.pi / 180
```

is equivalent to:

```
x = R * math.radians(longitude)
y = R * math.radians(latitude)
```

I recommend using `math.radians()` because it makes your intention clearer.

Expected output

For Jezero's approximate coordinates `(18.38°, 77.58°)`:

Projected X · 4,601,000 m

Projected Y · 1,089,000 m

Approximate values, rounded.

Next step: Connect everything

We now have all the pieces:

1. Latitude/longitude → projected X/Y.
2. X/Y → raster row/column.
3. Row/column → elevation.

Your next exercise is to write:

```
def get_elevation_from_latlon(dataset, latitude, longitude):
    # Convert coordinates
    # Retrieve elevation
```

Reuse your existing functions. Don't rewrite their internal logic.

One important detail: our conversion assumes the same spherical equirectangular convention as the MOLA raster. We'll verify the coordinate reference system carefully before relying on it for actual route planning.

You:

dataset = rasterio.open("mola.tif")

def is_inside_raster(dataset, row, col): return 0 <= row < dataset.height and 0 <= col < dataset.width

def get_pixel_position(dataset, x, y): row, col = dataset.index(x,y) if(is_inside_raster(dataset, row, col)): return (row, col)

```
  return None
```

def get_elevation_from_pixel(dataset, row, col): if(is_inside_raster(dataset, row, col)): window = Window(col_off=col, row_off=row, width=1, height=1)

```
      data = dataset.read(1, window=window, masked=True)
      elevation = data[0, 0]

      if(np.ma.is_masked(elevation)):
          return None

      return elevation

  return None
```

def get_elevation(dataset, x, y): position = get_pixel_position(dataset, x, y)

```
  if position is None:
      return None

  row, col = position
  elevation = get_elevation_from_pixel(dataset, row, col)

  return elevation
```

import math

def mars_latlon_to_xy(latitude, longitude): R = 3396190

```
  x = R * math.radians(longitude)
  y = R * math.radians(latitude)

  return x, y
```

def get_elevation_from_latlon(dataset, latitude, longitude): x, y = mars_latlon_to_xy(dataset, latitude, longitude) elevation = get_elevation(dataset, x, y)

```
  return elevation
```

ChatGPT:

You're very close! The overall architecture is correct, and you've successfully connected all the functions.

There's just **one small bug** in your final function.

1. The issue

You wrote:

```
x, y = mars_latlon_to_xy(dataset, latitude, longitude)
```

But your conversion function is defined as:

```
def mars_latlon_to_xy(latitude, longitude):
```

It accepts two arguments, not three.

So remove `dataset` from the call:

```
x, y = mars_latlon_to_xy(latitude, longitude)
```

2. Your corrected final function

```
def get_elevation_from_latlon(dataset, latitude, longitude):
    x, y = mars_latlon_to_xy(latitude, longitude)
    elevation = get_elevation(dataset, x, y)

    return elevation
```

3. Test it

Try:

```
print(get_elevation_from_latlon(dataset, 18.38, 77.58))
```

Compare this result with your earlier Jezero pixel elevation, approximately `-2645`.

If they differ, don't immediately assume the conversion is wrong. We'll investigate the CRS and coordinate conventions.

What you've built

Latitude / Longitude

↓

mars_latlon_to_xy()

Projected X / Y

↓

get_pixel_position()

Raster Row / Column

↓

get_elevation_from_pixel()

Elevation

Next milestone: We'll start extracting a small terrain patch around a selected Mars location. That patch will eventually become the foundation for calculating slope and building your route cost grid.

You:

-2645

ChatGPT:

Excellent! (Coordinate-to-elevation pipeline working)

Your Jezero test returned `-2645`, matching the earlier pixel-based result.

That means your complete pipeline is working:

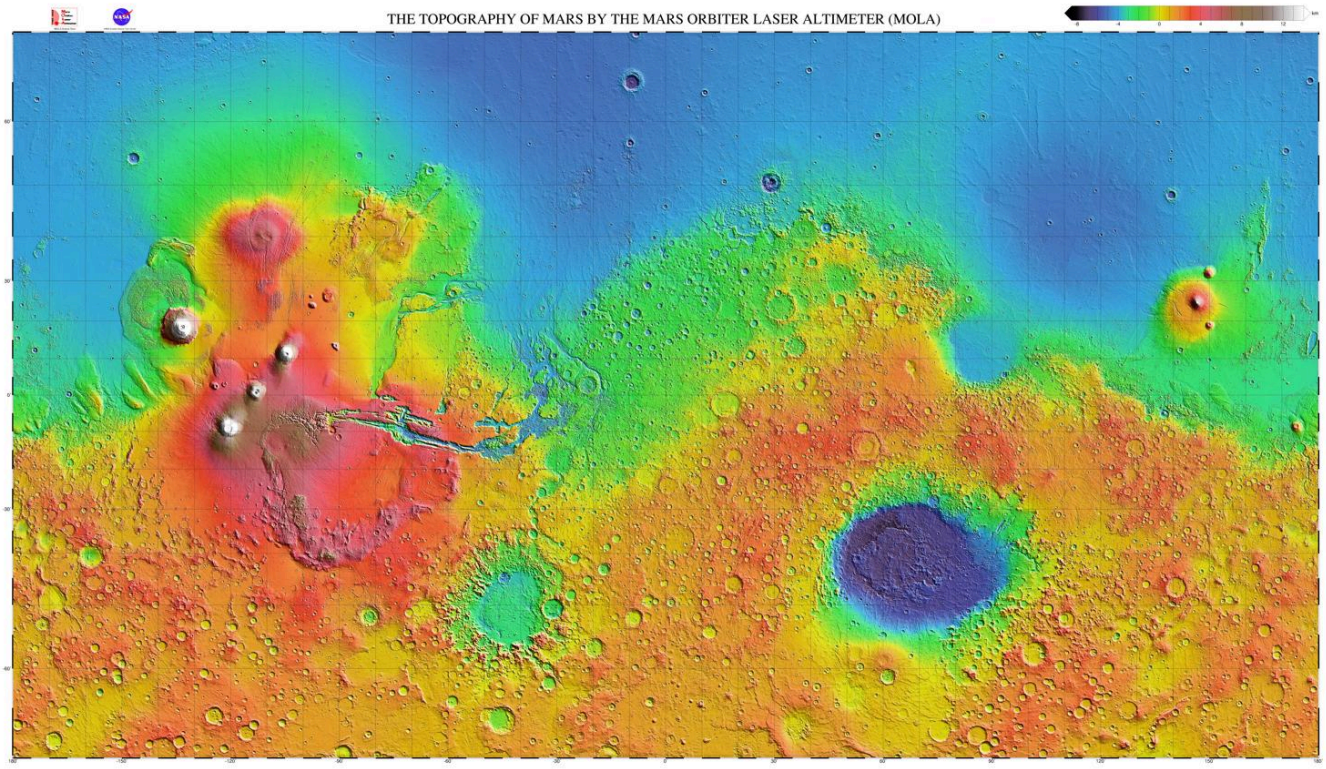
Latitude/Longitude → X/Y → Row/Column → Elevation

You've now built your first useful MOLA data-access function.

Next milestone: Extract a terrain patch

A route planner needs more than one elevation value. It needs to inspect the terrain surrounding a location.

For example, we could extract a **101 × 101 pixel patch** around Jezero.



At approximately 463 metres per pixel, that patch covers roughly 46.8 km in each direction.

Before coding, one question:

If the centre pixel is at:

```
row = 9167
col = 32970
```

And we want a 101 × 101 patch centred on it, how many pixels should we extend from the centre in each direction?

(Remember, 101 is an odd number.)

You:

aroud 4.5 or 5

ChatGPT:

You're thinking in the right direction, but let's distinguish **pixels** from kilometres.

For a 101 × 101 patch:

$$\frac{101 - 1}{2} = 50$$

So we extend **50 pixels in each direction** from the centre.

50 pixels

In each direction: north, south, east and west.

- 50 pixels north
- 50 pixels south
- 50 pixels east
- 50 pixels west
- 1 centre pixel

At approximately 463 metres per pixel:

$$50 \times 463 \approx 23.15 \text{ km}$$

So the patch extends approximately **23.15 km from the centre** in each direction.

Your next exercise

We have:

```
center_row = 9167
center_col = 32970
patch_size = 101
```

How would you calculate the starting row and starting column?

Hint: The starting position is 50 pixels before the centre in both dimensions.

Try writing just those two calculations.

You:

I didn't understand the patch thing

ChatGPT:

No worries. Let's slow down. The patch is actually a very simple idea.

1. Imagine the MOLA dataset as a giant grid

Your MOLA raster contains:

- 23,040 rows
- 46,080 columns

Each cell (pixel) stores an elevation value.

Imagine a tiny part of that grid:

A small terrain grid

C

Each square represents one pixel

The green square represents the centre pixel. The surrounding squares are neighbouring terrain pixels.

Every pixel has:

- A row number.
- A column number.
- An elevation value.

For example:

```
row = 9167
col = 32970
```

This identifies one particular location on Mars.

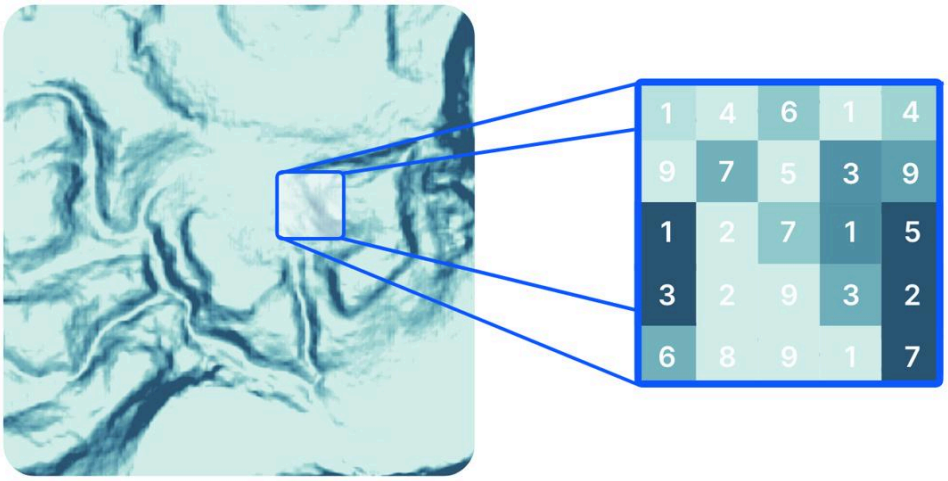
2. What is a terrain patch?

Suppose you want to plan a rover route around that location.

Looking at only one elevation value isn't enough. You need to examine the surrounding terrain.

So we extract a smaller square from the giant raster.

That smaller square is called a **patch**.



Think of it like selecting a small square area from a giant photograph.

3. Why 101 × 101?

Imagine selecting:

- 50 pixels to the left.
- 50 pixels to the right.
- 50 pixels above.
- 50 pixels below.
- The centre pixel itself.

That gives:

$$50 + 1 + 50 = 101$$

Therefore:

$$101 \times 101 = 10,201 \text{ pixels}$$

Instead of processing the entire 1-billion-pixel MOLA dataset, we process just 10,201 pixels around the location.

4. Let's check your understanding

Suppose our centre pixel is:

```
center_row = 100
center_col = 200
```

We want a patch extending 5 pixels in every direction.

What will be the starting row?

Hint: Subtract 5 from the centre row.

No need to write code yet. Just give me the number.

You:

95

ChatGPT:

Exactly! **95** is correct.

Because:

$$100 - 5 = 95$$

Now let's calculate the starting column.

```
center_row = 100
center_col = 200

start_row = center_row - 5  # 95
start_col = center_col - 5  # ?
```

What is `start_col`?

You:

195

ChatGPT:

Correct! Both starting coordinates are now clear.

```
start_row = 100 - 5  # 95
start_col = 200 - 5  # 195
```

Now let's complete the patch boundaries

We want 5 pixels on each side, plus the centre.

Boundary	Calculation	Result
Starting row	100 – 5	95
Starting column	200 – 5	195
Ending row (exclusive)	100 + 5 + 1	106
Ending column (exclusive)	200 + 5 + 1	206

Why add 1? Python's slicing excludes the ending index.

So this extracts exactly 11 × 11 pixels:

```
patch = data[95:106, 195:206]
```

Next question: Why do you think we use `106` instead of `105` as the ending row?

You:

because index 106 means upto 105

ChatGPT:

Exactly! You've got it.

Python slicing includes the starting index but excludes the ending index.

```
data[95:106]
```

Includes:

`95, 96, 97, ... 104, 105`

That's 11 rows.

The same applies to columns.

Now let's apply this to Rasterio

One important thing: we don't need to load the entire raster into memory to extract a patch. We can use `Window`.

For our example:

```
from rasterio.windows import Window

window = Window(
    col_off=195,
    row_off=95,
    width=11,
    height=11
)

patch = dataset.read(1, window=window, masked=True)
```

Notice the order:

- `col_off` = starting column.
- `row_off` = starting row.
- `width` = number of columns.
- `height` = number of rows.

And the resulting NumPy array has shape `(11, 11)`.

Your next exercise

For Jezero:

```
center_row = 9167
center_col = 32970
patch_size = 101
```

What should the following values be?

```
start_row = ?
start_col = ?
```

Hint: A 101-pixel patch extends 50 pixels on either side of its centre.

You:

Ok btw just asking how far are we to finish it

ChatGPT:

Honestly, we're making good progress. But we're still in the **terrain-data foundation**, not yet in the actual pathfinding algorithm.

For Marswalk, here's where we stand:

Marswalk routing progress

Phase 1 — MOLA data access · (Almost complete)

- Read raster metadata
- Understand rows, columns and pixels
- Convert coordinates to raster positions
- Retrieve elevation values
- Extract terrain patches — in progress

Phase 2 — Terrain analysis · (Upcoming)

- Calculate slope from elevation data.
- Build a slope map.
- Create terrain movement costs.

Phase 3 — Pathfinding · (Upcoming)

- Understand grid neighbours.
- Implement A*.
- Reconstruct the safest-cost route.

Phase 4 — Backend integration · (Upcoming)

- Build FastAPI endpoints.
- Connect the frontend.
- Display calculated routes on Leaflet.

Phase 5 — Optimization and validation · (Upcoming)

- Improve route performance.
- Validate terrain constraints.
- Optimize route geometry and display.

How far are we?

My rough estimate:

Area	Progress
Rasterio fundamentals	90%
Terrain extraction	In progress
Terrain analysis	0%
A* implementation	0%
Backend integration	0%
Frontend integration	0%

These are learning milestones, not measured project completion percentages.

How much is left?

I estimate around **10–15 focused learning and implementation sessions** to reach a basic working end-to-end route planner, assuming we maintain our current pace.

A genuinely reliable Mars rover route planner will require considerably more validation. MOLA's approximately 463 m pixel spacing is suitable for regional planning, but cannot establish rover-scale hazard safety.

My recommendation: Don't rush into A*. First finish terrain extraction, then learn slope calculation. Once you understand how terrain becomes a movement cost, A* will make much more sense.

We're building the foundation properly rather than just copying a pathfinding algorithm.

You:

Ok back to learning. Since patch is the no of cells we need, so 101 -1 / 2 = 50. -1 so that we exclude the center and find the rest of the expansion

---

ChatGPT:

Yes! You've understood the main idea. There is just one small clarification.

For an odd-sized patch, the formula is:

$$\text{Expansion} = \frac{\text{Patch size} - 1}{2}$$

For a 101 × 101 patch:

$$\frac{101 - 1}{2} = 50$$

Why subtract 1?

Because the centre pixel is already included. We distribute the remaining 100 pixels equally on both sides:

- 50 pixels before the centre.
- 1 centre pixel.
- 50 pixels after the centre.

So the centre is **not excluded**. It is included exactly once.

Now calculate the actual Jezero patch boundaries

```
center_row = 9167
center_col = 32970
patch_size = 101

expansion = (patch_size - 1) // 2

start_row = center_row - expansion
start_col = center_col - expansion
```

Your exercise: What values do you expect for `start_row` and `start_col`?

Take a shot at it!

You:

9117 and 32920

ChatGPT:

Exactly right! Both calculations are correct.

- `start_row = 9167 - 50 = 9117`
- `start_col = 32970 - 50 = 32920`

Now let's calculate the window dimensions.

```
from rasterio.windows import Window

window = Window(
    col_off=start_col,
    row_off=start_row,
    width=patch_size,
    height=patch_size
)

patch = dataset.read(1, window=window, masked=True)
```

Your extracted patch should have:

```
print(patch.shape)
# (101, 101)
```

One important concept before we continue

What happens if the centre pixel is too close to the edge of the raster?

For example, suppose:

```
center_row = 10
expansion = 50
```

Then:

```
start_row = center_row - expansion
# -40
```

Question: Why could this cause a problem when extracting the terrain patch?

Think it through before answering.

You:

Out of bounds can happen

ChatGPT:

Exactly! The patch would extend beyond the raster boundary.

In our example:

```
center_row = 10
expansion = 50

start_row = 10 - 50  # -40
```

A negative starting row is outside the raster.

Rasterio windows can represent areas extending beyond dataset boundaries, but reading them without careful handling can produce unexpected shapes or require boundless reading. For our first implementation, we'll simply reject incomplete patches.

Next step: Validate the patch boundaries

Before reading a patch, we need to check four conditions:

1. `start_row` must be at least 0.
2. `start_col` must be at least 0.
3. `start_row + patch_size` must not exceed `dataset.height`.
4. `start_col + patch_size` must not exceed `dataset.width`.

Your exercise

Write a function:

```
def is_valid_patch(dataset, start_row, start_col, patch_size):
    # Return True if the entire patch fits inside the raster
    # Otherwise return False
```

Try implementing these four conditions yourself.

We'll use this function to prevent invalid terrain patches before extracting them.

You:

def is_valid_patch(dataset, start_row, start_col, patch_size): expansion = (patch_size - 1) / 2 return 0 <= start_row - expansion < dataset.width and 0 <= start_col - expansion < dataset.height

---

ChatGPT:

Good attempt! You've understood that we need boundary checks, but there are **three issues** in your implementation.

Let's work through them.

1. Don't calculate expansion again

Your function receives `start_row` and `start_col`. These are already the starting positions of the patch.

So we don't need:

```
expansion = (patch_size - 1) / 2
```

2. Row and column dimensions are swapped

Remember:

- Rows are checked against `dataset.height`.
- Columns are checked against `dataset.width`.

Your current code uses the opposite dimensions.

3. Check the ending boundaries too

This is the most important part.

Suppose:

```
start_row = 100
patch_size = 101
```

The patch occupies rows 100 through 200.

We need to ensure the patch's ending position doesn't exceed the raster.

The ending index is exclusive, just like Python slicing.

Let's fix it together

Complete these four conditions:

```
def is_valid_patch(dataset, start_row, start_col, patch_size):

    return (
        # Condition 1: starting row
        0 <= start_row

        # Condition 2: starting column
        and ...

        # Condition 3: ending row
        and ...

        # Condition 4: ending column
        and ...
    )
```

Your task: Fill in the three missing conditions.

Hint: The ending row is `start_row + patch_size`, and the ending column is `start_col + patch_size`.

You:

def is_valid_patch(dataset, start_row, start_col, patch_size):

```
return (
    # Condition 1: starting row
    0 <= start_row - patch_size < dataset.height

    # Condition 2: starting column
    and 0 <= start_col - patch_size < dataset.width

    # Condition 3: ending row
    and 0 <= start_row + patch_size < dataset.height

    # Condition 4: ending column
    and 0 <= start_col + patch_size < dataset.width
)
```

ChatGPT:

Good effort! You've got the right idea of checking both starting and ending boundaries. However, there's one important mathematical correction.

You're subtracting `patch_size` from the starting positions:

```
0 <= start_row - patch_size < dataset.height
```

But `start_row` is already the starting boundary. We should check it directly.

Let's examine the four conditions

Condition	Correct expression
Starting row	`0 <= start_row`
Starting column	`0 <= start_col`
Ending row	`start_row + patch_size <= dataset.height`
Ending column	`start_col + patch_size <= dataset.width`

Notice that we use `<=` for the ending boundaries because the ending index is exclusive.

Your corrected function

```
def is_valid_patch(dataset, start_row, start_col, patch_size):
    return (
        0 <= start_row
        and 0 <= start_col
        and start_row + patch_size <= dataset.height
        and start_col + patch_size <= dataset.width
    )
```

Let's test your understanding

Suppose:

```
dataset.height = 23040
start_row = 22950
patch_size = 101
```

The ending row is:

$$22950 + 101 = 23051$$

Is this patch valid? Why or why not?

Remember, the raster has 23,040 rows, indexed from 0 to 23,039.

You:

Invalid

ChatGPT:

Correct! (Invalid patch)

Because:

$$22950 + 101 = 23051$$

But the raster height is only 23,040 rows.

The patch would extend beyond the raster boundary.

You've now learned three important concepts:

1. How to calculate a terrain patch around a centre pixel.
2. How Rasterio's `Window` extracts that patch.
3. How to validate patch boundaries before extraction.

Next step: Build the patch extraction function

Let's combine what you've learned.

Your task is to write:

```
def extract_terrain_patch(dataset, center_row, center_col, patch_size):
    # 1. Calculate expansion
    # 2. Calculate start_row and start_col
    # 3. Validate the patch
    # 4. Create a Rasterio Window
    # 5. Read the patch and return it
```

Reuse your existing `is_valid_patch()` function.

Don't worry about getting everything right on the first attempt. We'll build it together.

You:

def extract_terrain_patch(dataset, center_row, center_col, patch_size): # 1. Calculate expansion expansion = (patch_size - 1) / 2 # 2. Calculate start_row and start_col start_row = center_row - expansion start_col = center_col - expansion # 3. Validate the patch valid = is_valid_patch(dataset, start_row, start_col, patch_size) if(valid): window = Window(col_off = start_col, row_off = start_row, width = patch_size height = patch_size)

```
    patch = rasterio.read(1, window=window, masked=True)
    return patch

return None
# 4. Create a Rasterio Window
# 5. Read the patch and return it
```

ChatGPT:

Great work! You've assembled the entire extraction workflow yourself. The logic is almost correct. I found **three small issues** to fix.

1. Use integer expansion

You wrote:

```
expansion = (patch_size - 1) / 2
```

Python's `/` returns a float. Rasterio window offsets should be integers for our pixel-aligned window.

Use:

```
expansion = (patch_size - 1) // 2
```

2. Missing comma

Your `Window` constructor needs a comma after `width`:

```
width=patch_size,
height=patch_size
```

3. Read from the dataset, not Rasterio

You wrote:

```
patch = rasterio.read(1, window=window, masked=True)
```

But `rasterio` is the library. Your opened dataset is the object that reads the raster.

Use:

```
patch = dataset.read(1, window=window, masked=True)
```

Your corrected function

```
def extract_terrain_patch(dataset, center_row, center_col, patch_size):
    expansion = (patch_size - 1) // 2

    start_row = center_row - expansion
    start_col = center_col - expansion

    valid = is_valid_patch(dataset, start_row, start_col, patch_size)

    if valid:
        window = Window(
            col_off=start_col,
            row_off=start_row,
            width=patch_size,
            height=patch_size
        )

        patch = dataset.read(1, window=window, masked=True)
        return patch

    return None
```

Now test it

```
patch = extract_terrain_patch(dataset, 9167, 32970, 101)

if patch is not None:
    print("Patch shape:", patch.shape)
    print("Centre elevation:", patch[50, 50])
else:
    print("Invalid patch")
```

Expected results

- Patch shape: `(101, 101)`
- Centre elevation: approximately `-2645`

One important question

Why do we use:

```
patch[50, 50]
```

instead of:

```
patch[9167, 32970]
```

Think about the difference between the original raster coordinates and the extracted patch's local coordinates.

You:

We are looking up on a smaller portion not the whole so the rows and cols have been reduced to 101

ChatGPT:

Exactly! You've understood it.

The extracted patch is a new, smaller NumPy array. Its indexing starts from zero, independently of the original raster.

The original raster:

```
patch centre = (9167, 32970)
```

The extracted patch:

```
patch centre = (50, 50)
```

Here's the relationship:

$$\text{Local row} = \text{Global row} - \text{Start row}$$

$$\text{Local col} = \text{Global col} - \text{Start col}$$

For Jezero:

- Start row = 9117
- Start column = 32920
- Centre row = 9167
- Centre column = 32970

Therefore, the local centre is `(50, 50)`.

One more important concept: Window transform

Our patch now has its own local coordinate system.

Rasterio provides:

```
patch_transform = dataset.window_transform(window)
```

This lets us convert local patch pixel positions back into projected Mars coordinates.

We'll need this later when calculating slope and generating route geometry.

Your next exercise: Run your extraction function and show me:

```
print(patch.shape)
print(patch[50, 50])
print(np.ma.min(patch))
print(np.ma.max(patch))
```

Then we'll move into the exciting part: **calculating terrain slope from MOLA elevation data.**

You:

(101, 101) -2645 -2888 -1465 alienll

ChatGPT:

Excellent! Your terrain patch extraction is working.

Let's interpret your results:

Measurement	Result
Patch dimensions	101 × 101
Centre elevation	-2645
Minimum elevation	-2888
Maximum elevation	-1465

This means the extracted area contains elevations ranging from -2888 to -1465 (assuming the dataset's elevation units are metres).

The elevation difference is:

$$-1465 - (-2888) = 1423$$

So there's a 1,423-metre elevation range within this patch.

That's interesting terrain data for our route planner!

Next lesson: What is slope?

Elevation tells us how high or low a location is.

Slope tells us **how steep the terrain is**.

> Interactive content unavailable: AsyncImageGroup

Imagine a rover travelling between two neighbouring pixels:

- Pixel A elevation: 1000 m
- Pixel B elevation: 1100 m
- Horizontal distance: 463 m

The elevation difference is 100 m.

Slope percentage is:

$$\text{Slope} = \frac{\text{Elevation difference}}{\text{Horizontal distance}} \times 100$$

For our example:

$$\frac{100}{463} \times 100 \approx 21.6\%$$

That is a relatively steep incline.

Your first slope exercise

Suppose:

- Pixel A = 500 m
- Pixel B = 550 m
- Horizontal distance = 500 m

What is the slope percentage?

No coding yet. Let's make sure the mathematics is clear first.